

**ГУАП**

**КАФЕДРА № 43**

**ОТЧЕТ**

**ЗАЩИЩЕН С ОЦЕНКОЙ**

**ПРЕПОДАВАТЕЛЬ**

**Скобцов Ю.А.**

---

должность, уч. степень,  
звание

---

подпись, дата

---

инициалы, фамилия

**ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 3**

Решение задач комбинаторной оптимизации с помощью генетических алгоритмов  
на примере задачи укладки рюкзака

Вариант 4

по курсу: Эволюционные методы проектирования программно-информационных систем

**РАБОТУ ВЫПОЛНИЛ**

**СТУДЕНТ гр. №** 4333К

---

подпись, дата

**Могилатов С.И.**

---

инициалы, фамилия

**Санкт-Петербург 2026**

## СОДЕРЖАНИЕ

|                                                   |    |
|---------------------------------------------------|----|
| ВВЕДЕНИЕ.....                                     | 3  |
| 1 ЗАДАНИЕ И ПОСТАНОВКА ЗАДАЧИ .....               | 4  |
| 1.1 Индивидуальное задание.....                   | 4  |
| 1.2 Математическая постановка .....               | 4  |
| 2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ.....                     | 5  |
| 2.1 Двоичное кодирование .....                    | 5  |
| 2.2 Генетические операторы и восстановление ..... | 5  |
| 2.3 Критерии оценки .....                         | 5  |
| 3 РЕАЛИЗАЦИЯ И РЕЗУЛЬТАТЫ .....                   | 6  |
| 3.1 Структура программы .....                     | 6  |
| 3.2 Результаты базовых запусков .....             | 6  |
| 4 ИССЛЕДОВАНИЕ ПАРАМЕТРОВ.....                    | 8  |
| 4.1 Влияние мощности популяции .....              | 8  |
| 4.2 Влияние вероятности кроссовера.....           | 8  |
| 4.3 Влияние вероятности мутации .....             | 9  |
| 5 ВЫВОДЫ.....                                     | 11 |
| 6 ОТВЕТ НА КОНТРОЛЬНЫЙ ВОПРОС .....               | 12 |
| СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ .....            | 13 |
| Приложение А. Листинг алгоритма .....             | 14 |
| Приложение Б. Исходные данные.....                | 17 |
| Приложение В. Сценарий экспериментов.....         | 18 |

## **ВВЕДЕНИЕ**

Генетические алгоритмы позволяют искать решения в больших дискретных пространствах, используя имитацию естественного отбора. В лабораторной работе исследуется задача 0/1-рюкзака, в которой необходимо выбрать набор предметов максимальной стоимости при ограниченной вместимости.

Цель работы - реализовать генетический алгоритм с двоичным кодированием для задач укладки рюкзака, сравнить результат с точным оптимумом, исследовать влияние основных параметров и проанализировать алгоритм восстановления недопустимых решений.

В работе рассмотрены тест P04 простой сложности и набор 7 повышенной сложности для варианта 4. Приведены постановка задачи, теоретические сведения, описание программы, результаты экспериментов, графики сходимости и ответ на контрольный вопрос.

# 1 ЗАДАНИЕ И ПОСТАНОВКА ЗАДАЧИ

## 1.1 Индивидуальное задание

Согласно методическим указаниям для варианта 4 требуется использовать двоичное кодирование решения. Для задания простой сложности выбирается тестовый набор 4 (P04), для задания повышенной сложности - набор 7. Необходимо реализовать генетический алгоритм, сравнить решение с оптимальным, построить графики сходимости при разных параметрах, проанализировать время работы и точность, а также ответить на контрольный вопрос 4.

**Таблица 1 - Параметры индивидуального задания**

| Показатель         | Значение |
|--------------------|----------|
| Вариант            | 4        |
| Кодирование        | двоичное |
| Простая задача     | P04      |
| Повышенная задача  | набор 7  |
| Контрольный вопрос | 4        |

## 1.2 Математическая постановка

Пусть  $x_i$  принимает значение 0 или 1 и показывает, выбран ли предмет  $i$ . Тогда требуется максимизировать стоимость  $F(x) = \sum(p_i x_i)$  при ограничении  $\sum(w_i x_i) \leq C$ . Для P04 вместимость  $C$  равна 50, а эталонное решение имеет хромосому (1, 0, 0, 1, 0, 0, 0), суммарный вес 50 и стоимость 107.

Для набора 7 повышенной сложности используется 50 предметов и вместимость 12828. Точный эталонный результат, рассчитанный динамическим программированием, равен 16763 при весе 12820.

**Таблица 2 - Базовые параметры генетического алгоритма**

| Параметр               | P04          | Набор 7      |
|------------------------|--------------|--------------|
| Размер популяции       | 60           | 100          |
| Число поколений        | 250          | 400          |
| Вероятность кроссовера | 0,85         | 0,85         |
| Вероятность мутации    | 1/n          | 0,02         |
| Метод кроссовера       | двухточечный | двухточечный |
| Элитизм                | 2            | 2            |
| Число seed             | 5            | 5            |

## **2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ**

### **2.1 Двоичное кодирование**

При двоичном кодировании хромосома имеет длину  $n$  и состоит из нулей и единиц. Позиция гена соответствует номеру предмета. Такое представление непосредственно задает подмножество предметов, однако стандартные операции кроссовера и мутации могут привести к превышению вместимости рюкзака.

### **2.2 Генетические операторы и восстановление**

В программе используется турнирный отбор, двухточечный кроссовер, побитовая мутация и элитизм. После формирования хромосомы выполняется жадное восстановление: из перегруженного решения удаляется выбранный предмет с минимальным отношением стоимости к весу. Процедура повторяется до получения допустимого решения.

### **2.3 Критерии оценки**

Качество особи оценивается суммарной стоимостью восстановленного решения. В экспериментах дополнительно измеряются суммарный вес, ошибка относительно точного оптимума, процент точности и время выполнения. Фиксация seed позволяет сравнивать варианты параметров на воспроизводимых запусках.

## 3 РЕАЛИЗАЦИЯ И РЕЗУЛЬТАТЫ

### 3.1 Структура программы

Файл `data.py` содержит проверяемые исходные данные P04 и набора 7. В `knapsack_ga.py` реализованы оценка и восстановление особей, точный динамический эталон, турнирный отбор, двухточечный кроссовер, мутация, элитизм и основной цикл генетического алгоритма. Сценарий `run_experiments.py` выполняет базовые серии и параметрическое исследование, сохраняет CSV/JSON и строит графики.

### 3.2 Результаты базовых запусков

Таблица 3 - Сравнение с точным оптимумом

| Набор   | Точный profit | Вес   | Лучший ГА | Средний ГА | Точность, % | Среднее время, с |
|---------|---------------|-------|-----------|------------|-------------|------------------|
| P04     | 107           | 50    | 107       | 107.0      | 100.00      | 0.080            |
| Набор 7 | 16763         | 12820 | 16763     | 16752.8    | 99.94       | 0.634            |

Для P04 генетический алгоритм во всех пяти запусках получил оптимальную стоимость 107 и вес 50. Для набора 7 лучшая серия достигла точного результата 16763 при среднем значении 16752.8, что соответствует средней точности 99.94%. Все сохраненные решения удовлетворяют ограничению вместимости.

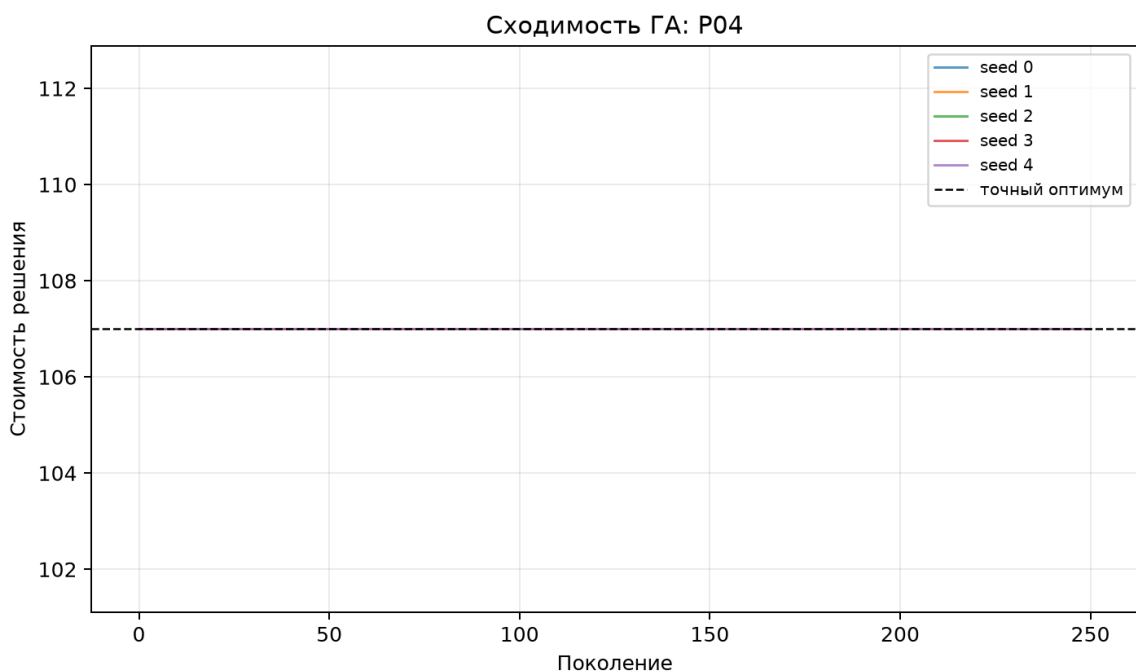


Рисунок 1 - Сходимость генетического алгоритма для P04

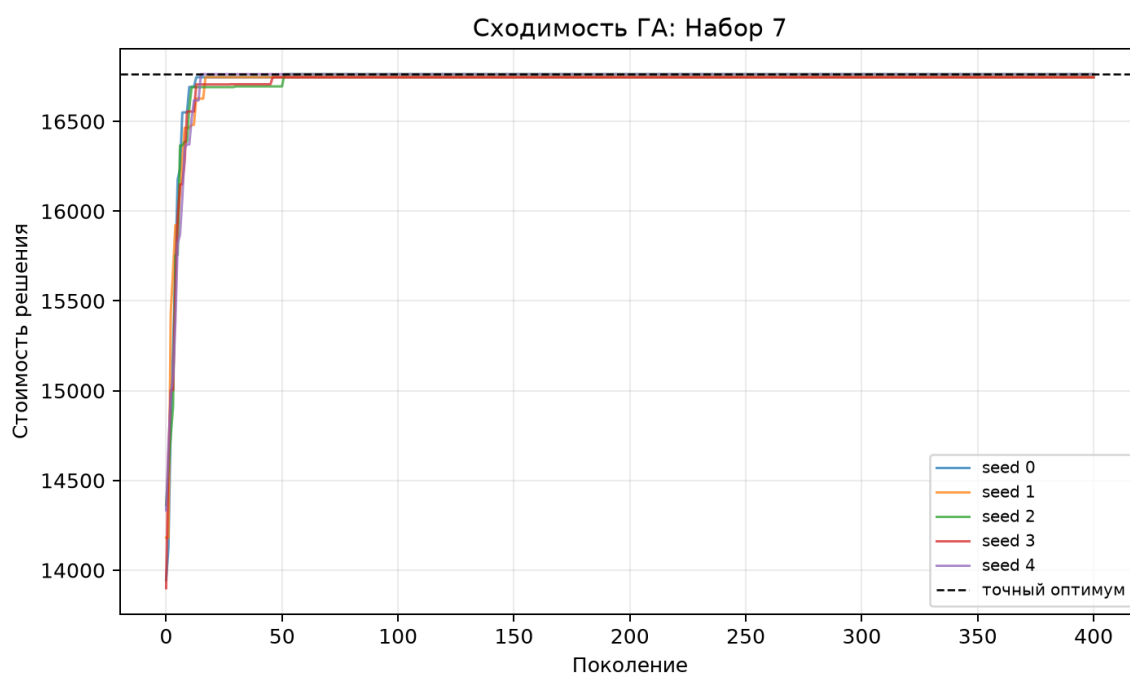


Рисунок 2 - Сходимость генетического алгоритма для набора 7

## 4 ИССЛЕДОВАНИЕ ПАРАМЕТРОВ

### 4.1 Влияние мощности популяции

При увеличении популяции растет разнообразие хромосом и количество вычислений в каждом поколении. Для набора 7 сравнивались 20, 60 и 100 особей. Увеличение популяции не гарантирует монотонного роста точности, но снижает риск преждевременной потери перспективных комбинаций.

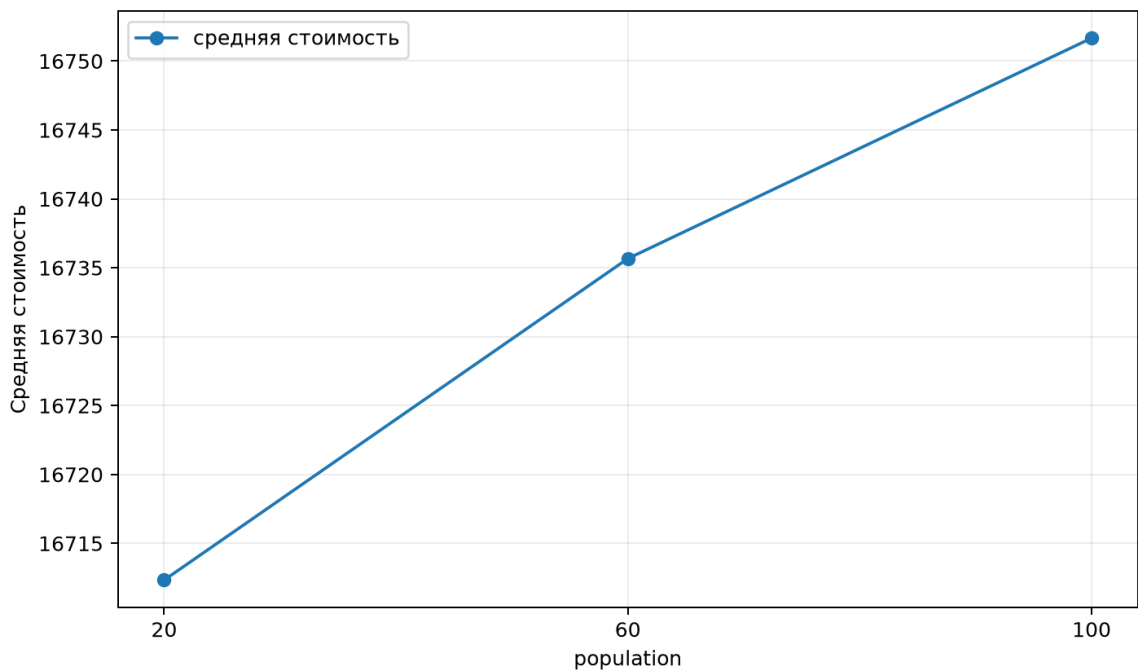


Рисунок 3 - Влияние мощности популяции на среднюю стоимость

### 4.2 Влияние вероятности кроссовера

Вероятность кроссовера определяет долю потомков, полученных путем обмена фрагментами родительских хромосом. При малой вероятности сильнее сохраняются родительские решения, а при значении, близком к единице, пространство поиска исследуется интенсивнее. В работе рассмотрены значения 0,60, 0,85 и 1,00.

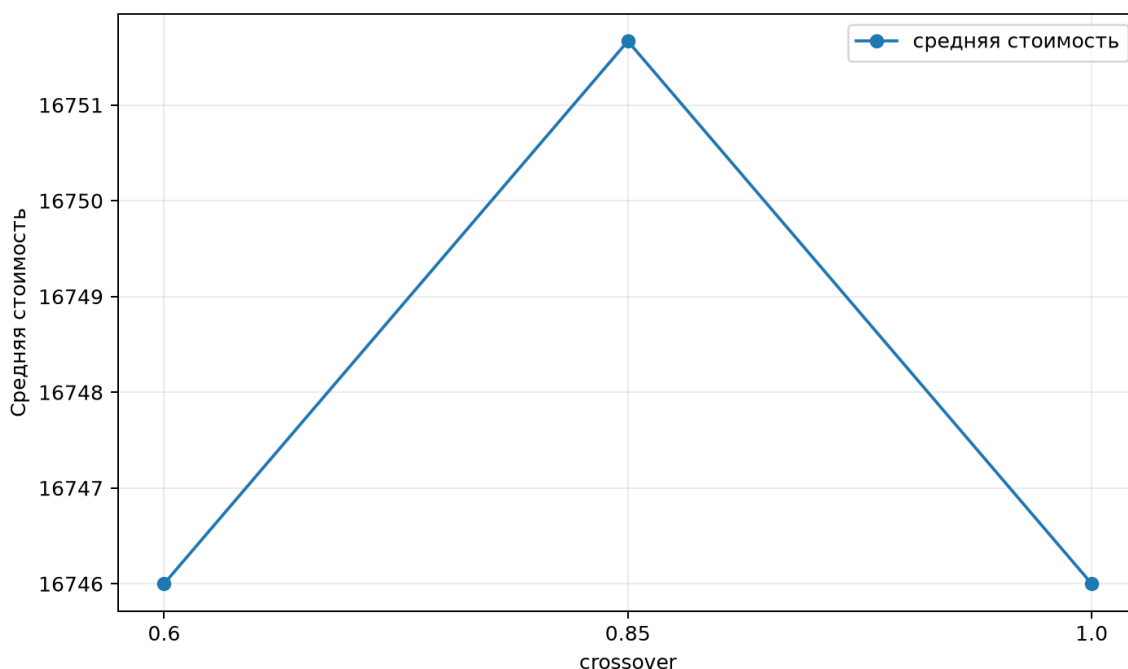


Рисунок 4 - Влияние вероятности кроссовера

### 4.3 Влияние вероятности мутации

Мутация поддерживает разнообразие популяции и позволяет выходить из локальных областей поиска. Слишком малая вероятность ограничивает разнообразие, а слишком большая разрушает найденные комбинации. В работе рассмотрены значения 0,005, 0,02 и 0,05.

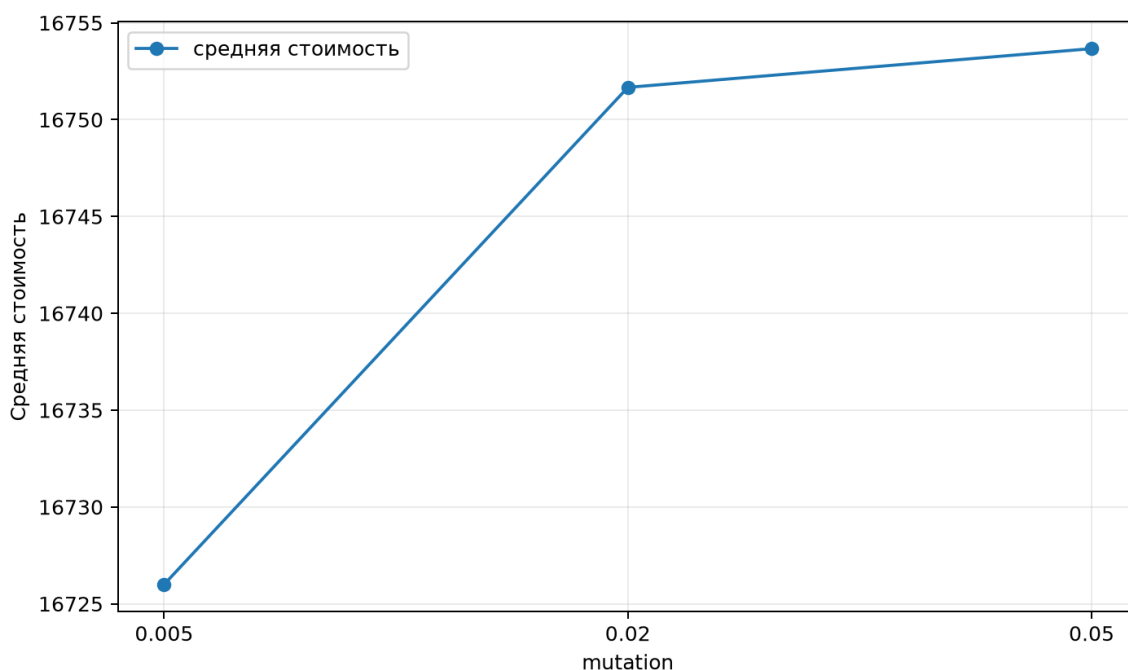


Рисунок 5 - Влияние вероятности мутации

**Таблица 4 - Результаты параметрического исследования набора 7**

| <b>Фактор</b> | <b>Значение</b> | <b>Средняя стоимость</b> | <b>Точность, %</b> | <b>Время, с</b> |
|---------------|-----------------|--------------------------|--------------------|-----------------|
| population    | 20              | 16712.3                  | 99.70              | 0.117           |
| population    | 60              | 16735.7                  | 99.84              | 0.374           |
| population    | 100             | 16751.7                  | 99.93              | 0.632           |
| crossover     | 0.6             | 16746.0                  | 99.90              | 0.613           |
| crossover     | 0.85            | 16751.7                  | 99.93              | 0.643           |
| crossover     | 1.0             | 16746.0                  | 99.90              | 0.631           |
| mutation      | 0.005           | 16726.0                  | 99.78              | 0.588           |
| mutation      | 0.02            | 16751.7                  | 99.93              | 0.629           |
| mutation      | 0.05            | 16753.7                  | 99.94              | 0.680           |

## 5 ВЫВОДЫ

В ходе лабораторной работы реализован генетический алгоритм с двоичным кодированием для задачи 0/1-рюкзака. Для устранения недопустимых решений применено жадное восстановление по отношению стоимости к весу.

На тесте P04 алгоритм во всех базовых запусках нашел эталонное решение со стоимостью 107 и весом 50. На наборе 7 повышенной сложности лучшая серия достигла точного динамического оптимума 16763 при допустимом весе 12820. Это подтверждает работоспособность двоичного кодирования и оператора восстановления.

Параметрическое исследование показало, что размер популяции и вероятности генетических операторов задают компромисс между разнообразием поиска, точностью и временем. Результаты экспериментов сохранены в CSV и JSON, а исходный код приведен в приложениях.

## **6 ОТВЕТ НА КОНТРОЛЬНЫЙ ВОПРОС**

Алгоритм восстановления преобразует недопустимую особь в допустимую, если после скрещивания или мутации общий вес выбранных предметов превышает вместимость рюкзака. Для этого из текущего решения последовательно удаляются выбранные предметы по заданному правилу. В данной работе удаляется предмет с минимальным отношением стоимости к весу. После устранения перегрузки решение снова проверяется, и полученная допустимая хромосома используется для оценки и дальнейшей эволюции.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Лабораторная работа № 3. Решение задач комбинаторной оптимизации с помощью генетических алгоритмов на примере задачи укладки рюкзака: методические указания.
2. Вирсански Э. Генетические алгоритмы на Python / пер. с англ. А. А. Слинкина. - М.: ДМК Пресс, 2020. - 286 с.
3. Скобцов Ю. А. Вычислительный интеллект. - СПб.: ГУАП, 2022. - 138 с.
4. ГУАП. Нормативная документация для учебного процесса. - URL: <https://guap.ru/c/regdocs/docs/uch> (дата обращения: 27.09.2026).
5. Burkardt J. KNAPSACK\_01: Data for the 01 Knapsack Problem. - URL: [https://people.sc.fsu.edu/~jburkardt/datasets/knapsack\\_01/](https://people.sc.fsu.edu/~jburkardt/datasets/knapsack_01/) (дата обращения: 27.09.2026).

## Приложение А. Листинг алгоритма

```
"""Binary genetic algorithm for 0/1 knapsack benchmarks."""

from __future__ import annotations

from dataclasses import dataclass
import random
from time import perf_counter
from typing import Iterable

from data import KnapsackProblem

@dataclass(frozen=True)
class Evaluation:
    chromosome: tuple[int, ...]
    weight: int
    profit: int

@dataclass(frozen=True)
class GARun:
    problem: str
    best: Evaluation
    history: tuple[int, ...]
    elapsed_seconds: float
    seed: int
    parameters: dict[str, float | int | str]

def _validate_chromosome(chromosome: Iterable[int], problem: KnapsackProblem) -> tuple[int, ...]:
    result = tuple(int(gene) for gene in chromosome)
    if len(result) != len(problem.weights):
        raise ValueError("chromosome length must equal the number of items")
    if any(gene not in (0, 1) for gene in result):
        raise ValueError("binary chromosome genes must be 0 or 1")
    return result

def repair(chromosome: Iterable[int], problem: KnapsackProblem) -> Evaluation:
    """Repair an overweight chromosome by removing the least profitable density items."""
    genes = list(_validate_chromosome(chromosome, problem))
    weight = sum(gene * item_weight for gene, item_weight in zip(genes, problem.weights))
    while weight > problem.capacity:
        selected = [i for i, gene in enumerate(genes) if gene]
        if not selected:
            break
        index = min(
            selected,
            key=lambda i: (problem.profits[i] / problem.weights[i], problem.profits[i], -i)
            if problem.weights[i]
            else (-float("inf"), problem.profits[i], -i),
        )
        genes[index] = 0
        weight -= problem.weights[index]
    profit = sum(gene * item_profit for gene, item_profit in zip(genes, problem.profits))
    return Evaluation(tuple(genes), weight, profit)

def evaluate(chromosome: Iterable[int], problem: KnapsackProblem) -> Evaluation:
    return repair(chromosome, problem)
```

```

def exact_optimum(problem: KnapsackProblem) -> Evaluation:
    """Solve the benchmark exactly with 0/1 dynamic programming for validation."""
    capacity = problem.capacity
    values = [0] * (capacity + 1)
    choices: list[bytearray] = [bytearray(capacity + 1) for _ in problem.weights]
    for index, (weight, profit) in enumerate(zip(problem.weights, problem.profits)):
        for current in range(capacity, weight - 1, -1):
            candidate = values[current - weight] + profit
            if candidate > values[current]:
                values[current] = candidate
                choices[index][current] = 1

    chromosome = [0] * len(problem.weights)
    current = capacity
    for index in range(len(problem.weights) - 1, -1, -1):
        if choices[index][current]:
            chromosome[index] = 1
            current -= problem.weights[index]
    return evaluate(chromosome, problem)

def _tournament(population: list[Evaluation], rng: random.Random, size: int = 3) -> Evaluation:
    contestants = [population[rng.randrange(len(population))]] for _ in range(size)
    return max(contestants, key=lambda individual: individual.profit)

def _crossover(
    first: tuple[int, ...],
    second: tuple[int, ...],
    rng: random.Random,
    method: str,
) -> tuple[tuple[int, ...], tuple[int, ...]]:
    if len(first) < 2:
        return first, second
    if method == "two_point" and len(first) >= 3:
        left, right = sorted(rng.sample(range(1, len(first)), 2))
        child_a = first[:left] + second[left:right] + first[right:]
        child_b = second[:left] + first[left:right] + second[right:]
    else:
        point = rng.randrange(1, len(first))
        child_a = first[:point] + second[point:]
        child_b = second[:point] + first[point:]
    return child_a, child_b

def _mutate(chromosome: tuple[int, ...], rng: random.Random, probability: float) -> tuple[int, ...]:
    return tuple(1 - gene if rng.random() < probability else gene for gene in chromosome)

def genetic_algorithm(
    problem: KnapsackProblem,
    *,
    population_size: int = 60,
    generations: int = 250,
    crossover_probability: float = 0.85,
    mutation_probability: float | None = None,
    crossover_method: str = "two_point",
    elite_size: int = 2,
    seed: int = 0,
) -> GARun:
    if population_size < 2 or generations < 1:
        raise ValueError("population_size must be >= 2 and generations must be >= 1")
    if not 0 <= crossover_probability <= 1 or not 0 <= elite_size < population_size:

```

```

        raise ValueError("invalid crossover probability or elite size")
    if mutation_probability is None:
        mutation_probability = 1 / len(problem.weights)
    if not 0 <= mutation_probability <= 1:
        raise ValueError("mutation probability must be between 0 and 1")
    if crossover_method not in {"one_point", "two_point"}:
        raise ValueError("unsupported crossover method")

    rng = random.Random(seed)
    started = perf_counter()
    population = [
        evaluate((rng.randrange(2) for _ in problem.weights), problem)
        for _ in range(population_size)
    ]
    best = max(population, key=lambda individual: individual.profit)
    history = [best.profit]

    for _ in range(generations):
        ordered = sorted(population, key=lambda individual: individual.profit, reverse=True)
        next_population = ordered[:elite_size]
        while len(next_population) < population_size:
            parent_a = _tournament(population, rng)
            parent_b = _tournament(population, rng)
            child_a, child_b = parent_a.chromosome, parent_b.chromosome
            if rng.random() < crossover_probability:
                child_a, child_b = _crossover(child_a, child_b, rng, crossover_method)
            child_a = _mutate(child_a, rng, mutation_probability)
            next_population.append(evaluate(child_a, problem))
            if len(next_population) < population_size:
                child_b = _mutate(child_b, rng, mutation_probability)
                next_population.append(evaluate(child_b, problem))
        population = next_population
        current = max(population, key=lambda individual: individual.profit)
        if current.profit > best.profit:
            best = current
        history.append(best.profit)

    return GARun(
        problem=problem.name,
        best=best,
        history=tuple(history),
        elapsed_seconds=perf_counter() - started,
        seed=seed,
        parameters={
            "population_size": population_size,
            "generations": generations,
            "crossover_probability": crossover_probability,
            "mutation_probability": mutation_probability,
            "crossover_method": crossover_method,
            "elite_size": elite_size,
        },
    )

```

## Приложение Б. Исходные данные

```
"""Benchmark data for laboratory work 3, variant 4."""

from __future__ import annotations

from dataclasses import dataclass

@dataclass(frozen=True)
class KnapsackProblem:
    name: str
    capacity: int
    weights: tuple[int, ...]
    profits: tuple[int, ...]

    def __post_init__(self) -> None:
        if len(self.weights) != len(self.profits):
            raise ValueError("weights and profits must have the same length")
        if self.capacity < 0 or any(w < 0 for w in self.weights):
            raise ValueError("capacity and weights must be non-negative")

P04 = KnapsackProblem(
    name="P04",
    capacity=50,
    weights=(31, 10, 20, 19, 4, 3, 6),
    profits=(70, 20, 39, 37, 7, 5, 10),
)

SET7 = KnapsackProblem(
    name="Набор 7",
    capacity=12828,
    weights=(
        981, 119, 419, 758, 152, 489, 40, 669, 765, 574,
        876, 314, 696, 595, 580, 457, 840, 945, 475, 665,
        61, 702, 648, 994, 822, 285, 386, 669, 23, 462,
        169, 118, 59, 769, 130, 248, 391, 872, 81, 450,
        550, 884, 820, 864, 279, 416, 359, 885, 958, 151,
    ),
    profits=(
        324, 151, 651, 73, 536, 366, 58, 508, 38, 434,
        70, 91, 425, 827, 124, 224, 628, 948, 578, 397,
        977, 47, 859, 290, 145, 118, 309, 817, 181, 582,
        639, 373, 548, 63, 60, 206, 681, 428, 315, 586,
        454, 300, 795, 699, 245, 575, 526, 876, 730, 288,
    ),
)

PROBLEMS = {P04.name: P04, SET7.name: SET7}
```

## Приложение В. Сценарий экспериментов

```
"""Run reproducible experiments for laboratory work 3."""

from __future__ import annotations

import csv
import json
from pathlib import Path
import statistics
import sys

import matplotlib.pyplot as plt

ROOT = Path(__file__).resolve().parent
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from data import P04, SET7, KnapsackProblem
from knapsack_ga import GARun, exact_optimum, genetic_algorithm

RESULTS = ROOT / "results"
BASELINE = {
    P04.name: {"population_size": 60, "generations": 250, "seeds": range(5)},
    SET7.name: {"population_size": 100, "generations": 400, "seeds": range(5)},
}

def run_once(problem: KnapsackProblem, seed: int, **overrides: int | float | str) -> GARun:
    parameters = dict(BASELINE[problem.name])
    parameters.pop("seeds")
    parameters.update(overrides)
    return genetic_algorithm(problem, seed=seed, **parameters)

def as_result(problem: KnapsackProblem, run: GARun, optimum: int, label: str) -> dict:
    return {
        "problem": problem.name,
        "label": label,
        "seed": run.seed,
        "profit": run.best.profit,
        "weight": run.best.weight,
        "capacity": problem.capacity,
        "error": optimum - run.best.profit,
        "accuracy_percent": 100 * run.best.profit / optimum if optimum else 100.0,
        "elapsed_seconds": run.elapsed_seconds,
        "chromosome": list(run.best.chromosome),
        "parameters": run.parameters,
    }

def summarize(rows: list[dict]) -> dict:
    profits = [row["profit"] for row in rows]
    accuracies = [row["accuracy_percent"] for row in rows]
    return {
        "runs": len(rows),
        "best_profit": max(profits),
        "mean_profit": statistics.mean(profits),
        "stdev_profit": statistics.stdev(profits) if len(profits) > 1 else 0.0,
        "mean_accuracy_percent": statistics.mean(accuracies),
        "mean_time_seconds": statistics.mean(row["elapsed_seconds"] for row in rows),
    }
```

```

def save_csv(path: Path, rows: list[dict]) -> None:
    fields = [
        "problem", "label", "seed", "profit", "weight", "capacity", "error",
        "accuracy_percent", "elapsed_seconds",
    ]
    with path.open("w", newline="", encoding="utf-8") as stream:
        writer = csv.DictWriter(stream, fieldnames=fields)
        writer.writeheader()
        writer.writerows({field: row[field] for field in fields} for row in rows)

def plot_convergence(problem: KnapsackProblem, runs: list[GARun], optimum: int) -> None:
    plt.figure(figsize=(8, 4.8))
    for run in runs:
        plt.plot(run.history, linewidth=1.2, alpha=0.75, label=f"seed {run.seed}")
    plt.axhline(optimum, color="black", linestyle="--", linewidth=1.0, label="точный оптимум")
    plt.title(f"Сходимость ГА: {problem.name}")
    plt.xlabel("Поклоение")
    plt.ylabel("Стоимость решения")
    plt.grid(alpha=0.25)
    plt.legend(fontsize=8)
    plt.tight_layout()
    plt.savefig(RESULTS / f"convergence_{problem.name.replace(' ', '_').lower()}.png", dpi=180)
    plt.close()

def parameter_study(problem: KnapsackProblem, optimum: int) -> list[dict]:
    rows: list[dict] = []
    base = BASELINE[problem.name]
    studies = [
        ("population", "population_size", [20, 60, 100]),
        ("crossover", "crossover_probability", [0.60, 0.85, 1.00]),
        ("mutation", "mutation_probability", [0.005, 0.02, 0.05]),
    ]
    for label, parameter, values in studies:
        for value in values:
            for seed in range(3):
                overrides = {parameter: value}
                if parameter == "population_size":
                    overrides["generations"] = base["generations"]
                run = run_once(problem, seed, **overrides)
                rows.append(as_result(problem, run, optimum, f"{label}={value}"))
    return rows

def plot_parameter_study(rows: list[dict], problem: KnapsackProblem, label: str, filename: str) -> None:
    selected = [row for row in rows if row["problem"] == problem.name and row["label"].startswith(label + "=")]
    values = []
    means = []
    accuracies = []
    for row in selected:
        value = row["label"].split("=", 1)[1]
        if value not in values:
            values.append(value)
            group = [item for item in selected if item["label"] == row["label"]]
            means.append(statistics.mean(item["profit"] for item in group))
            accuracies.append(statistics.mean(item["accuracy_percent"] for item in group))
    x = list(range(len(values)))
    fig, axis = plt.subplots(figsize=(8, 4.8))
    axis.plot(x, means, marker="o", label="средняя стоимость")
    axis.set_xticks(x, values)
    axis.set_xlabel(label)
    axis.set_ylabel("Средняя стоимость")
    axis.grid(alpha=0.25)

```

```

axis.legend()
fig.tight_layout()
fig.savefig(RESULTS / filename, dpi=180)
plt.close(fig)

def main() -> None:
    RESULTS.mkdir(exist_ok=True)
    problems = (P04, SET7)
    exact = {problem.name: exact_optimum(problem) for problem in problems}
    baseline_rows: list[dict] = []
    all_runs: dict[str, list[GARun]] = {}
    summaries: dict[str, dict] = {}

    for problem in problems:
        runs = [run_once(problem, seed) for seed in BASELINE[problem.name]["seeds"]]
        all_runs[problem.name] = runs
        rows = [as_result(problem, run, exact[problem.name].profit, "baseline") for run in runs]
        baseline_rows.extend(rows)
        summaries[problem.name] = {
            "exact": {
                "profit": exact[problem.name].profit,
                "weight": exact[problem.name].weight,
                "chromosome": list(exact[problem.name].chromosome),
            },
            "baseline": summarize(rows),
        }
        plot_convergence(problem, runs, exact[problem.name].profit)

    study_rows = []
    for problem in problems:
        study_rows.extend(parameter_study(problem, exact[problem.name].profit))
    save_csv(RESULTS / "baseline.csv", baseline_rows)
    save_csv(RESULTS / "parameter_study.csv", study_rows)
    for label, filename in (
        ("population", "parameter_population.png"),
        ("crossover", "parameter_crossover.png"),
        ("mutation", "parameter_mutation.png"),
    ):
        plot_parameter_study(study_rows, SET7, label, filename)

    summaries["parameter_study"] = {
        "rows": len(study_rows),
        "best": max(study_rows, key=lambda row: row["profit"]),
    }
    with (RESULTS / "results_summary.json").open("w", encoding="utf-8") as stream:
        json.dump({"problems": summaries}, stream, ensure_ascii=False, indent=2)
    print(json.dumps({"problems": summaries}, ensure_ascii=False, indent=2))

if __name__ == "__main__":
    main()

```